TD 6 – SVM en Python

Intelligence Artificielle Hybride et Apprentissage Automatique

Exercice 1 - Premier SVM sans texte

Objectif : Comprendre le fonctionnement minimal d'un SVM sur des données numériques très simples. Étant donné les données suivantes :

```
X = [
    [5, 0],
    [6, 0],
    [7, 1],
    [15, 3],
    [18, 4],
    [20, 5]
]

Y = ["neutre", "neutre", "neutre", "expressif", "expressif", "expressif"]
```

Implémentez un programme Python afin de :

- Importer SVC depuis sklearn.svm.
- Créer un modèle SVM.
- Entraîner le modèle sur X et Y.
- Prédire la classe de [8, 0] puis de [17, 4].
- Afficher les prédictions.

Questions de réflexion

- Que représentent ici les deux nombres de chaque ligne ?
- Pourquoi ce jeu de données est-il facile à classer ?

Exercice 2 - Question ou affirmation ?

Objectif : Classer des phrases à partir de deux indices linguistiques. Imaginez que nous encodons chaque phrase par :

- présence d'un point d'interrogation : 0 ou 1
- présence d'un mot interrogatif : 0 ou 1

Étant donné les données suivantes :

```
X = [
    [1, 1],  # "Pourquoi viens-tu ?"
    [1, 0],  # "Tu viens ?"
    [0, 0],  # "Tu viens."
    [0, 0],  # "Je suis là."
    [1, 1],  # "Comment ça va ?"
    [0, 0]   # "Il fait beau."
]

y = ["question", "question", "affirmation", "affirmation", "question", "affirmation"]
```

Implémentez un programme Python afin de :

- Entraîner un SVM sur les données.
- Tester le modèle sur [1, 0], [0, 1] et [0, 0].
- Comparer les résultats obtenus.

Questions de réflexion

- Une phrase peut-elle être reconnue comme une question sans mot interrogatif ?
- Pourquoi le cas [0, 1] est-il intéressant ?

Exercice 3 - Registre formel ou informel

Objectif : utiliser des indices de registre très simples dans de petits messages. Chaque message est décrit par :

- présence d'abréviations : 0 ou 1
- présence de "vous" : 0 ou 1
- présence d'émoticônes : 0 ou 1

Étant donné les données suivantes :

```
X = [
    [1, 0, 1],  # "slt :)"
    [1, 0, 0],  # "bjr"
    [0, 1, 0],  # "Pouvez-vous m'aider ?"
    [0, 1, 0],  # "Je vous remercie."
    [1, 0, 1],  # "cc :)"
    [0, 1, 0]   # "Veuillez trouver..."
]

y = ["informel", "informel", "formel", "formel", "informel", "formel"]
```

Implémentez un programme Python afin de :

- Entraîner un SVM.
- Prédire la catégorie de [1, 0, 0] puis de [0, 1, 1].
- Expliquer ce que représentent les trois variables.

Question de réflexion

- Un message peut-il mélanger des indices formels et informels ?

Exercice 4 - Avis positif ou négatif

Objectif : Faire une classification binaire très simple à partir de comptes de mots. Nous décrivons chaque avis avec :

- nombre de mots positifs
- nombre de mots négatifs

```
X = [
    [3, 0],
    [2, 0],
    [0, 3],
    [0, 2],
    [2, 1],
    [1, 2]
]

y = ["positif", "positif", "negatif", "negatif", "positif", "negatif"]
```

Implémentez un programme Python afin de :

- Créer et entraîner un SVM.
- Prédire la classe de [3, 1], [0, 4] et [1, 1].
- Commenter les résultats.

Questions de réflexion

- Pourquoi [1, 1] est-il plus ambigu ?
- Quel effet a l'équilibre entre les mots positifs et négatifs ?

Exercice 5 - Passer au texte avec CountVectorizer

Objectif : Découvrir une représentation automatique des textes avant l'apprentissage SVM. CountVectorizer transforme chaque texte en vecteur de fréquences de mots.

```
texts = [
    "bonjour madame je vous remercie",
    "salut ça va",
    "veuillez recevoir mes salutations",
    "coucou merci :)",
    "je vous prie d accepter",
    "hello :)"
]

labels = ["formel", "informel", "formel", "informel", "formel", "informel"]
```

Implémentez un programme Python afin de :

- Transformer les textes avec CountVectorizer.
- Entraîner un SVM à partir des vecteurs obtenus.
- Prédire la catégorie de « bonjour », « coucou ça va » et « je vous remercie ».

Questions de réflexion

- Quels mots semblent porter le plus d'information ?
- Pourquoi « bonjour » peut-il être interprété de plusieurs façons ?